



Universidad
Tecnológica
del Perú

Universidad Tecnológica del Perú

Guía De Laboratorio 07

Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje para Instituciones Educativas

INTEGRANTES – GRUPO 6:

Rivera Castañeda, Jesús Martín	- U21213646
Quispe Fernández, Alonso	- U20208103
Alanya Guevara, Ben Andre	- U21315933
Ninatanta Fernandez, Renato Ventura	- U21205210

ASIGNATURA:

Curso Integrador II: Software

DOCENTE:

Barturen Huaman, Renzo Renato

SECCIÓN:

31738

Lima – Perú

ÍNDICE

1. Resultado y alcance del incremento	3
1.1 Producto a desarrollar	3
2. Correspondencia con el sílabo	3
3. Arquitectura del incremento	4
3.1 Matriz de acceso por endpoint	4
3.2 Flujo seguro del envío de un intento	4
3.3 Estructura de carpetas	5
4. Conceptos de seguridad aplicados	5
4.1 Autenticación y autorización no son equivalentes	5
4.2 Contraseña, hash y token	6
4.3 Alcance del token local	6
4.4 Token en memoria y CSRF	6
4.5 Concurrencia e integridad	6
4.6 Trazabilidad sin fuga de datos	7
5. Modelo de amenazas	7
6. Preparación segura del entorno	8
Dependencias nuevas	8
7. Identidad y autenticación implementadas	9
8. Autorización, envío de intento y trazabilidad	9
9. Pruebas funcionales y de seguridad	10
Pruebas automatizadas	10
10. Riesgos conectados al incremento	11
11. Entregables y estado de evidencia (E0–E11)	12
12. Checklist de cierre	13

1. Resultado y alcance del incremento

Este avance corresponde a la Semana 7 del Curso Integrador II: Software y da continuidad al incremento de catálogo y contenidos construido en la Semana 6 sobre ESTUD-IA. El objetivo de esta etapa es convertir una operación sensible del sistema en un flujo autenticado, autorizado, transaccional y trazable: que un Alumno inicie sesión y envíe el intento de un ejercicio de forma protegida, en lugar de que cualquier visitante pueda hacerlo sin identificarse.

1.1 Producto a desarrollar

- Extensión de la tabla users con rol y public_id, y migración de personal_access_tokens publicada desde Sanctum.
- Contraseñas generadas localmente y almacenadas mediante el hash adaptativo de Laravel (Hash::make, bcrypt).
- Endpoint POST /api/v1/auth/login con respuesta genérica ante credenciales inválidas.
- Token de acceso de Sanctum con expiración de 15 minutos; secreto de la aplicación (APP_KEY) fuera de Git.
- Endpoint POST /api/v1/ejercicios/{id}/ autorizados solo para el rol Alumno, protegido con middleware auth:sanctum y verificación de rol.
- Envío atómico del intento con bloqueo pesimista, restricción UNIQUE y respuesta 409 ante conflicto.
- Interceptor de Axios, ruta protegida, formulario de inicio de sesión y envío real en React.
- Auditoría sin contraseñas ni tokens; correlación entre la respuesta HTTP y el evento registrado.

2. Correspondencia con el sílabo

Tema de la semana 7	Aplicación en ESTUD-IA	Evidencia verificable
Gestión de seguridad	Modelo de amenazas acotado al flujo iniciar sesión + enviar intento de ejercicio, con matriz control-prueba.	Matriz completada y decisiones justificadas.
Controles para el proyecto	Secretos fuera del repositorio (.env), mínimo privilegio por rol, validación de parámetros y errores genéricos.	Configuración, pruebas negativas y git diff.
Autenticación	Usuario persistente en MySQL, contraseña con Hash::make, Auth::attempt y login mediante Sanctum.	401 genérico, login correcto y hash no reversible.

Tema de la semana 7	Aplicación en ESTUD-IA	Evidencia verificable
Autorización	Middleware auth:sanctum y verificación de rol; el usuario autenticado identifica al alumno.	Alumno recibe 201; Docente recibe 403.
Cifrado y confidencialidad	Hash adaptativo para contraseñas, token de Sanctum validado por el framework y exigencia de HTTPS en producción.	Secreto fuera del repositorio y explicación hash/token.
Trazabilidad	Middleware de correlación (X-Correlation-ID) y tabla evento_auditoria sin datos sensibles.	Mismo identificador en respuesta y en la consulta de auditoría.
Defensa ante ataques	Throttle de login (RateLimiter de Laravel), Form Request con reglas mínimas, bloqueo de fila y CSP para la API.	429, 422, 409 y revisión de encabezados.
Mejores prácticas web	Token en memoria del Front-End, CORS explícito, respuestas JSON consistentes y pruebas.	Pruebas de React y build de producción.

3. Arquitectura del incremento

3.1 Matriz de acceso por endpoint

Solicitud	Acceso	Identidad usada	Control en servidor
GET cursos / contenidos	Público	Ninguna	Método GET y validación de parámetros.
POST login	Público y limitado	Correo y contraseña	Hash de Laravel, error genérico y throttle.
POST intentos de ejercicio	Alumno	Usuario autenticado (Sanctum)	Ruta, middleware de rol, bloqueo y UNIQUE.
Rutas administrativas futuras	Administrador	Usuario autenticado (Sanctum)	Regla /api/v1/admin/**.

3.2 Flujo seguro del envío de un intento

1. React adjunta el Bearer mediante el interceptor de Axios.
2. El middleware de correlación genera o valida X-Correlation-ID y el throttle limita intentos de login.
3. auth:sanctum valida el token y el middleware de rol confirma que el usuario sea Alumno.

4. El Service obtiene el id del usuario autenticado y bloquea la fila del ejercicio.
5. MySQL actualiza el ejercicio si corresponde, inserta el intento y registra el evento de auditoría.

El cuerpo de la solicitud contiene únicamente ejerciciold y respuesta. No acepta alumnold: si ese identificador viniera del navegador, un usuario podría intentar registrar un intento a nombre de otro alumno. El servicio obtiene la identidad desde el usuario autenticado por Sanctum y vuelve a aplicar el rol en el servidor; la ruta protegida de React solo mejora la navegación y nunca constituye la barrera real de autorización.

3.3 Estructura de carpetas

```
backend/
├── app/Http/Controllers/Api/AuthController.php
├── app/Http/Controllers/Api/IntentoEjercicioController.php
├── app/Http/Middleware/CorrelationIdMiddleware.php
├── app/Http/Middleware/EnsureRole.php
├── app/Http/Requests/LoginRequest.php
├── app/Http/Requests/EnviarIntentoRequest.php
├── app/Services/AuthService.php
├── app/Services/IntentoEjercicioService.php
├── app/Models/{User,Ejercicio,IntentoEjercicio,EventoAuditoria}.php
├── database/migrations/
│   ├── 2026_XX_XX_000003_add_rol_and_public_id_to_users_table.php
│   ├── 2026_XX_XX_000004_create_personal_access_tokens_table.php (Sanctum)
│   ├── 2026_XX_XX_000005_create_intento_ejercicio_table.php
│   └── 2026_XX_XX_000006_create_evento_auditoria_table.php
└── frontend/
    ├── src/features/auth/
    │   ├── AuthContext.jsx (sesión en memoria)
    │   ├── axiosInterceptor.js
    │   ├── ProtectedRoute.jsx
    │   └── LoginForm.jsx
```

4. Conceptos de seguridad aplicados

4.1 Autenticación y autorización no son equivalentes

Autenticar responde quién presenta la solicitud. Autorizar decide si esa identidad puede ejecutar una acción concreta sobre un recurso concreto. Un token válido no concede acceso total: el servidor debe verificar el rol y, cuando exista propiedad del recurso, comprobar también esa relación.

Situación	Resultado	Código
No existe token en una operación protegida	No autenticado	401

Situación	Resultado	Código
Token válido de Docente intenta enviar un intento	Autenticado pero no autorizado	403
Token de Alumno y ejercicio disponible	Operación autorizada	201
Token de Alumno y ejercicio ya con intento en curso	Autorizado, pero conflicto de dominio	409

4.2 Contraseña, hash y token

Mecanismo	Propósito	Reversible	Uso en esta práctica
Hash adaptativo (bcrypt)	Verificar una contraseña con sal y costo.	No	Hash::make genera el hash del usuario demo.
Cifrado (TLS)	Proteger confidencialidad con una clave.	Sí	TLS debe proteger las peticiones en producción.
Token de Sanctum	Identificar la sesión y su expiración sin exponer la contraseña.	No aplica	Validado por el framework contra personal_access_tokens.

El token de acceso puede inspeccionarse como una cadena opaca almacenada y hasheada en `personal_access_tokens`; no contiene la contraseña, el correo ni datos académicos sensibles del alumno. Sanctum valida su existencia, expiración y abilities antes de crear la identidad autenticada de Laravel.

4.3 Alcance del token local

Sanctum emite y valida el token con el mismo proceso Laravel durante el laboratorio, de forma razonable cuando un único Back-End controla ambas operaciones. Si en el futuro varias aplicaciones necesitaran validar la sesión de forma independiente, correspondería evaluar un proveedor OIDC externo, del mismo modo que la guía recomienda para un JWT local firmado con HS256.

4.4 Token en memoria y CSRF

React conserva el token únicamente en el estado de un `AuthContext`; no se escribe en `localStorage` ni `sessionStorage`. Al recargar la página se pierde la sesión, una decisión deliberada para reducir la persistencia ante un eventual XSS. El navegador no adjunta automáticamente el header `Authorization`, y las rutas `api/*` de Laravel ya excluyen el middleware de CSRF por la misma razón que expone la guía: CSRF protege credenciales que el navegador envía solas (cookies), no un bearer token añadido manualmente por el interceptor.

4.5 Concurrencia e integridad

Dos solicitudes pueden intentar enviar el mismo ejercicio casi al mismo tiempo. El Service bloquea la fila del ejercicio con `lockForUpdate()` dentro de una transacción hasta finalizar la operación; la segunda solicitud espera y luego encuentra el estado ya actualizado. Además, `uk_intento_ejercicio` garantiza como máximo un intento activo por alumno y ejercicio. El bloqueo controla el caso normal; la restricción es la última barrera de integridad, igual que en la guía.

4.6 Trazabilidad sin fuga de datos

Cada solicitud recibe X-Correlation-ID. El middleware acepta un valor externo únicamente si cumple un patrón seguro; en caso contrario genera un UUID. El evento de auditoría registra actor público, acción, resultado, recurso y correlación. No registra contraseñas, tokens, cabeceras completas ni el correo de un login fallido.

5. Modelo de amenazas

La matriz se completó antes de programar, trabajando solo con el flujo iniciar sesión → enviar intento y con datos sintéticos.

Amenaza	Control construido	Prueba esperada
Fuerza bruta	Cinco intentos por correo/IP en 60 s mediante <code>RateLimiter</code> ; error genérico.	El sexto POST login devuelve 429.
Enumeración de usuarios	Mismo 401 para correo inexistente o contraseña errónea.	El mensaje no cambia entre ambos casos.
Robo de credenciales almacenadas	Hash adaptativo (bcrypt) con sal; APP_KEY solo local.	password no coincide con el hash almacenado.
BOLA o IDOR	<code>alumnoid</code> no forma parte del cuerpo del intento; se usa el usuario autenticado.	Un body con <code>alumnoid</code> es ignorado o rechazado.
Token modificado o vencido	Sanctum valida el token contra <code>personal_access_tokens</code> y su expiración de 15 min.	Un token alterado o vencido devuelve 401.
Intento duplicado (doble envío)	Bloqueo de fila y restricción UNIQUE sobre <code>intento_ejercicio</code> .	Primera solicitud 201, segunda 409.
Inyección SQL	Eloquent con parámetros enlazados; formato de <code>ejercicioId</code> restringido en el Form Request.	Entrada maliciosa produce 422 sin alterar datos.
Fuga en logs	Eventos mínimos y correlación validada; contraseñas y tokens excluidos del log.	No aparecen password ni token en los registros.

Amenaza	Control construido	Prueba esperada
Acceso saltando la interfaz	ProtectedRoute e interceptor mejoran la experiencia; el servidor decide.	Llamar la API sin pasar por React sigue devolviendo 401/403.

6. Preparación segura del entorno

Se crea la rama desde el último commit funcional de la Semana 6. No se genera otro proyecto Laravel ni otro workspace de React. Se usan PHP 8.3 o superior, Composer, Laravel 11 o superior con Sanctum, Node 20 o superior, npm y MySQL 8.

```
git status --short
git switch -c feature/semana7-seguridad-intentos
php --version
composer --version
node --version
npm --version
```

Se copia .env.example como .env, se genera APP_KEY con php artisan key:generate y se definen dos contraseñas sintéticas de al menos 12 caracteres para las cuentas demo (Alumno y Docente). El archivo .env permanece ignorado por Git.

```
cd backend
cp .env.example .env
php artisan key:generate
# Definir DEMO_ALUMNO_PASSWORD y DEMO_DOCENTE_PASSWORD solo en .env
git status --short
git check-ignore .env
```

Comprueba: git check-ignore imprime .env y git status no lo muestra. No se ejecuta echo sobre los secretos ni se incluyen en capturas.

Dependencias nuevas

Dependencia	Responsabilidad
laravel/sanctum	Emisión y validación de tokens de acceso personal (bearer), sin estado en el servidor.
Middleware auth:sanctum	Cadena de autenticación equivalente al filtro de recurso OAuth2 de Spring Security.
RateLimiter (Illuminate\Cache)	Límite de intentos de login, equivalente al LoginThrottleFilter de la guía.

No se agrega una biblioteca JWT paralela. Sanctum ya resuelve emisión, validación y expiración del token; mantener una sola implementación reduce configuraciones incompatibles.

7. Identidad y autenticación implementadas

La tabla users, ya provista por Laravel, se extiende con public_id y rol (ENUM 'ALUMNO', 'DOCENTE', 'ADMINISTRADOR'), en lugar de crear una tabla app_user desde cero como en la guía: reutilizar la tabla por defecto del framework es más sostenible que duplicarla. La migración de Sanctum (personal_access_tokens) se publica con php artisan vendor:publish --tag=sanctum-migrations y se ejecuta junto con las demás.

- AuthController@login recibe correo y contraseña validados por LoginRequest, usa Auth::attempt (que internamente compara con Hash::check) y responde 401 genérico si falla, sin indicar si el correo existe.
- En caso de éxito, crea un token con \$user->createToken(name, ['role:'.\$user->rol])->plainTextToken y establece su expiración en 15 minutos mediante la configuración expiration de Sanctum, replicando el expiresIn de 900 segundos de la guía.
- El seeder de datos demo (DemoUserSeeder) crea las cuentas sintéticas Alumno y Docente solo cuando DEMO_USERS_ENABLED=true, variable que permanece en false por defecto.

Comprueba: la columna password comienza con el prefijo del algoritmo bcrypt (\$2y\$) y no contiene la contraseña local; el login válido devuelve accessToken y expiresIn: 900; el correo inexistente y la contraseña incorrecta devuelven el mismo 401.

```
curl -i -X POST http://localhost:8000/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"alumno@demo.example","password":"TU_PASSWORD_LOCAL"}
```

8. Autorización, envío de intento y trazabilidad

- EnviarIntentoRequest acepta únicamente ejerciciold (con formato restringido) y respuesta; nunca alumnold.
- IntentoEjercicioController obtiene el alumno mediante auth()->id(), nunca desde el cuerpo HTTP.
- IntentoEjercicioService carga el ejercicio con lockForUpdate() dentro de DB::transaction() y vuelve a comprobar que no exista un intento en curso del mismo alumno.
- Inserta intento_ejercicio y el evento INTENTO_REGISTRADO antes del commit, y devuelve 201 con el código público del intento.

En React, AuthContext conserva la respuesta del login únicamente en memoria; axiosInterceptor adjunta Authorization solo a las rutas /api y limpia la sesión ante un 401; ProtectedRoute redirige a /ingresar con un returnUrl cuando falta sesión, sin llamar a la navegación imperativa dentro del propio guard de ruta — la barrera real sigue estando en el Back-End.

Se ejecutan los cuatro escenarios de autorización: sin token la API responde 401; con token de Docente responde 403; con token de Alumno el primer envío responde 201; repetir el mismo ejercicio con Alumno responde 409. Los cuatro resultados corresponden a identidad, permiso y estado del dominio, sin confundir 401, 403 y 409.

9. Pruebas funcionales y de seguridad

Los tokens se preparan solo en la terminal, sin pegarlos en el informe, chat, issue, commit ni captura.

ID	Escenario	Esperado	Evidencia sin secretos
S01	GET catálogo sin token	200	Código, tiempo y cuerpo.
S02	POST intento sin token	401	Detalle del error y correlación.
S03	Login inválido	401	Mensaje genérico.
S04	Sexto login en menos de 60 s	429	Retry-After.
S05	Token con un carácter modificado	401	No mostrar el token.
S06	Docente intenta enviar un intento	403	Título de acceso denegado.
S07	Alumno envía un intento	201	Código del intento y ejercicio.
S08	Alumno repite el mismo ejercicio	409	Conflicto de dominio.
S09	ejerciciold con caracteres inválidos	422	Detalle de validación.
S10	Consultar resultados después de enviar	200 con el intento reflejado	Lista de resultados actualizada.
S11	Correlación del intento	Coincide	Header y fila auditada.
S12	Front-End sin sesión	Redirige	URL /ingresar y returnUrl.

Pruebas automatizadas

```
# Back-End
cd backend && php artisan test

# Front-End
```

```
cd ../frontend && npm test -- --watch=false && npm run build
```

Suite	Comportamiento principal
AuthServiceTest	Error genérico y auditoría de autenticación rechazada.
IntentoEjercicioServiceTest	Envío disponible y rechazo de un segundo intento concurrente.
DatabaseIntegrationTest	Migraciones y consulta real con MySQL cuando el servicio está disponible.
AuthContext / LoginForm (Vitest + RTL)	Sesión en memoria y cierre de sesión.
axiosInterceptor spec	Adjunta el token solo a rutas /api y limpia la sesión ante 401.
IntentoForm spec	Envío mínimo, respuesta 201 y traducción del 409.

Antes de cada commit se revisa que no queden secretos expuestos:

```
git status --short
git diff --check
git diff -- . '!*.lock' | grep -Ei 'password|secret|accessToken' || true
```

Una coincidencia no siempre es una filtración: nombres de variables y casos de prueba pueden aparecer; cada resultado se revisa manualmente y nunca se usan capturas que revelen el valor real.

10. Riesgos conectados al incremento

Se retoman los riesgos R-01 a R-12 registrados en los avances anteriores y se agregan dos riesgos propios de esta semana.

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador relacionado
R-13	Una contraseña o secreto de prueba podría filtrarse si el repositorio se hace público antes de limpiar el historial.	Commit que incluye .env o un valor con formato de secreto.	Mantener .env fuera de Git, generar secretos con herramientas locales y revisar con git diff --check antes de cada commit.	SLI-01, SLI-02

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador relacionado
R-14	Simplificar los roles a una sola columna en users podría dificultar permisos más granulares cuando el proyecto crezca (por ejemplo, un Docente con acceso parcial a varios cursos).	Requisito nuevo que exige permisos por recurso, no solo por rol.	Documentar la decisión como simplificación deliberada y evaluar un paquete de roles y permisos (por ejemplo spatie/laravel-permission) si el alcance lo exige.	DEL-01

11. Entregables y estado de evidencia (E0–E11)

ID	Entregable	Contenido mínimo	Estado
E0	Inicio controlado	Issue, rama feature/semana7-seguridad-intentos, commit base.	Pendiente
E1	Modelo de amenazas	Matriz completa de la Sección 5, con decisiones justificadas.	Completado (propuesta)
E2	Migraciones de identidad y auditoría	Extensión de users, personal_access_tokens, intento_ejercicio y evento_auditoria.	Pendiente
E3	Login	POST /api/v1/auth/login con Sanctum y error genérico.	Pendiente
E4	Autorización de intentos	Middleware de rol y POST /api/v1/ejercicios/{id}/intentos.	Pendiente
E5	Concurrencia e integridad	Bloqueo de fila y restricción UNIQUE verificados.	Pendiente
E6	Auditoría y correlación	Middleware de correlación y tabla evento_auditoria.	Pendiente
E7	Integración del Front-End	AuthContext, interceptor de Axios y ruta protegida.	Pendiente
E8	Reemplazo del envío simulado	Formulario de ejercicio conectado a la API real.	Pendiente
E9	Pruebas de seguridad	Matriz S01–S12 ejecutada y documentada.	Pendiente
E10	Pruebas automatizadas	Suite PHPUnit y pruebas de React.	Pendiente

ID	Entregable	Contenido mínimo	Estado
E11	Defensa	Presentación y Pull Request con revisión por pares.	Pendiente

12. Checklist de cierre

- Las contraseñas se almacenan únicamente como hash adaptativo, nunca en texto plano ni de forma reversible.
- El servidor vuelve a verificar el rol en cada endpoint protegido, sin confiar en la ruta protegida del Front-End.
- El token vive solo en memoria del Front-End y no se persiste en localStorage ni sessionStorage.
- Un envío duplicado queda controlado por bloqueo de fila y respaldado por una restricción UNIQUE.
- La auditoría registra acción, resultado y correlación sin contraseñas, tokens ni datos sensibles.
- Los secretos permanecen fuera del repositorio y se verificaron antes de cada commit.
- Los doce escenarios de la matriz de pruebas se ejecutaron y los códigos HTTP no se confunden entre sí.
- Los riesgos, el backlog y el Pull Request se actualizarán con la nueva evidencia.